Mlxtend Library

BSc. Nhat-Tan Bui

Software Engineering Lab (SELab),
University of Science - VNUHCM
Artificial Intelligence and Computer Vision Lab (AICV),
University of Arkansas, Fayetteville
https://tanbuinhat.github.io/

Content

- Introduction
- Preprocessing
- Apriori algorithm
- FP-Growth algorithm
- FP-Max algorithm
- Association rules

Mlxtend

- Documentation: http://rasbt.github.io/mlxtend/
- Source code: https://github.com/rasbt/mlxtend
- Python library of useful tools for the day-to-day data science tasks
- Main features: classifier, cluster, frequent patterns, plotting, feature extraction, regressor, etc.
- Installing
  - pip install mlxtend

Preprocessing

- Declare a Python list of transaction database

```
dataset = [['Bread', 'Chese', 'Juice'],
           ['Dill', 'Onion', 'Nutmeg', 'Kidney Beans', 'Eggs', 'Yogurt'],
           ['Milk', 'Apple', 'Kidney Beans', 'Eggs'],
           ['Eggs', 'Milk', 'Yogurt', 'Kidney Beans', 'Yogurt'],
           ['Corn', 'Onion', 'Onion', 'Kidney Beans', 'Ice cream', 'Biscuit']]
```

- Encode class for transaction data in Python lists

```
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)
```

Preprocessing

- Result

	Apple	Biscuit	Bread	Chese	Corn	Dill	Eggs	Ice cream	Juice	Kidney Beans	Milk	Nutmeg	Onion	Yogurt
0	False	False	True	True	False	False	False	False	True	False	False	False	False	False
1	False	False	False	False	False	True	True	False	False	True	False	True	True	True
2	True	False	False	False	False	False	True	False	False	True	True	False	False	False
3	False	False	False	False	False	False	True	False	False	True	True	False	False	True
4	False	True	False	False	True	False	False	True	False	True	False	False	True	False

Dataframe of boolean array of the input transactions, where the columns represent the unique items found in the input array in alphabetic order and the rows represent each transaction.

Apriori algorithm

- Apriori is a popular algorithm for extracting frequent itemsets with applications in association rule learning. The apriori algorithm has been designed to operate on databases containing transactions, such as purchases by customers of a store. An itemset is considered as "frequent" if it meets a user-specified support threshold. For instance, if the support threshold is set to 0.5 (50%), a frequent itemset is defined as a set of items that occur together in at least 50% of all transactions in the database.

Apriori algorithm

```
from mlxtend.frequent_patterns import apriori

apriori(df, min_support=0.5, use_colnames=True)
```

	support	itemsets
0	0.6	(Eggs)
1	0.8	(Kidney Beans)
2	0.6	(Eggs, Kidney Beans)

Since the database has 5 transactions, the ‘Eggs’ appears 3 times so the support will be 60%, the “Kidney Beans” appears 4 times so the support will be 80% and the combination “Eggs, Kidney Beans” appears 3 times so the support will be 60%

FP-growth algorithm

- FP-Growth is an algorithm for extracting frequent itemsets with applications in association rule learning that emerged as a popular alternative to the established Apriori algorighm.

- In particular, and what makes it different from the Apriori frequent pattern mining algorithm, FP-Growth is an frequent pattern mining algorithm that does not require candidate generation. Internally, it uses a so-called FP-tree (frequent pattern tree) datastrucure without generating the candidate sets explicitely, which makes is particularly attractive for large datasets.

FP-growth algorithm

```
from mlxtend.frequent_patterns import fpgrowth

fpgrowth(df, min_support=0.5, use_colnames=True)
```

	support	itemsets
0	0.8	(Kidney Beans)
1	0.6	(Eggs)
2	0.6	(Kidney Beans, Eggs)

FP-Max algorithm

- FP-Max is a variant of FP-Growth, which focuses on obtaining maximal itemsets. An itemset X is said to maximal if X is frequent and there exists no frequent super-pattern containing X. In other words, a frequent pattern X cannot be sub-pattern of larger frequent pattern to qualify for the definition maximal itemset.

FP-Max algorithm

```
from mlxtend.frequent_patterns import fpmax

fpmax(df, min_support=0.5, use_colnames=True)
```

	support	itemsets
0	0.6	(Kidney Beans, Eggs)

Association rules

- Rule generation is a common task in the mining of frequent patterns. An association rule is an implication expression of the form **X→Y**, where **X** and **Y** are disjoint itemsets. A more concrete example based on consumer behaviour would be **{Diapers}→{Beer}** suggesting that people who buy diapers are also likely to buy beer.
- To evaluate the "interest" of such an association rule, different metrics have been developed such as: *support, confidence, lift, leverage, conviction.*

Association rules

```
] from mlxtend.frequent_patterns import fpgrowth
  from mlxtend.frequent_patterns import association_rules

  frequent_itemsets = fpgrowth(df, min_support=0.5, use_colnames=True)
  association_rules(frequent_itemsets, metric="support", min_threshold=0.6)
```

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction
0	(Eggs)	(Kidney Beans)	0.6	0.8	0.6	1.00	1.25	0.12	inf
1	(Kidney Beans)	(Eggs)	0.8	0.6	0.6	0.75	1.25	0.12	1.6

Rules derived from the frequent itemsets only if the level of support is above the 60 percent threshold (min_threshold=0.6)